Semester	T.E. Semester VI Div A Batch 3 – CMPN
Subject	Machine Learning Lab
Subject Professor In-charge	Prof. Kavita Shirsat
Assisting Teachers	Prof. Kavita Shirsat
Laboratory	M313A

Student Name	Nikhil Patil	
Roll Number	23102A0057	
Grade and Subject Teacher's Signature		

Experiment Number	05
Experiment Title	**Implementation of Singular Value Decomposition (SVD) for Image Dataset**
Objectives (Skill Set / Knowledge Tested / Imparted)	Implementation of Singular Value Decomposition (SVD) for Image Dataset Compression and Feature Extraction

Implementation

1. Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files animal_ima..._dataset.zip
animal_image_dataset.zip(application/x-zip-compressed) - 45699071 bytes, last modified: 3/11/2026 - 100% done
Saving animal_image_dataset.zip to animal_image_dataset (1).zip
```

2. Read and Preprocess image

```
image_paths = []

for root, dirs, files in os.walk("dataset"):

    for file in files:

        if file.endswith(".jpg") or file.endswith(".png"):

            path = os.path.join(root, file)
            image_paths.append(path)

print("Total images found:", len(image_paths))
```

```
Total images found: 268
```

3. Display a sample image

```
plt.imshow(dataset[10], cmap='gray')
plt.title("Original Image")
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(127.5), np.float64(127.5), np.float64(-0.5))
```

4. Performed SVD

```
def hardcoded_svd(A):

    ATA = np.dot(A.T, A)

    eigenvalues, V = np.linalg.eig(ATA)

    idx = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[idx]
    V = V[:, idx]

    singular_values = np.sqrt(np.abs(eigenvalues))

    U = np.zeros((A.shape[0], len(singular_values)))

    for i in range(len(singular_values)):

        if singular_values[i] > 1e-10:

            U[:, i] = np.dot(A, V[:, i]) / singular_values[i]

    Sigma = np.zeros_like(A)

    for i in range(min(A.shape)):
        Sigma[i, i] = singular_values[i]

    return U, Sigma, V
```

5. Applied SVD to one image

```
A = dataset[10].astype(float)

U, Sigma, V = hardcoded_svd(A)

print("U shape:", U.shape)
print("Sigma shape:", Sigma.shape)
print("V shape:", V.shape)
```

```
U shape: (128, 128)
Sigma shape: (128, 128)
V shape: (128, 128)
```

6. Image reconstruction using k singular values

```
k = 40

Uk = U[:, :k]
Sigmak = Sigma[:k, :k]
Vk = V[:, :k]

reconstructed = np.dot(Uk, np.dot(Sigmak, Vk.T))
```

7. Compared Original and reconstructed image

```
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(A, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(reconstructed, cmap='gray')
plt.title("Reconstructed Image")
plt.axis("off")

plt.show()
```

8. Applied SVD to entire Dataset

```
reconstructed_dataset = []

k = 40

for img in dataset:

    A = img.astype(float)

    U, Sigma, V = hardcoded_svd(A)

    Uk = U[:, :k]
    Sigmak = Sigma[:k, :k]
    Vk = V[:, :k]

    reconstructed = np.dot(Uk, np.dot(Sigmak, Vk.T))

    reconstructed_dataset.append(reconstructed)

reconstructed_dataset = np.array(reconstructed_dataset)

print("Reconstructed dataset shape:", reconstructed_dataset.shape)
```

```
Reconstructed dataset shape: (20, 128, 128)
```

9. Dimensionality Reduction Calculation

```
original_dimension = 128 * 128

compressed_dimension = (128 * k) + k + (128 * k)

print("Original dimension:", original_dimension)
print("Compressed dimension:", compressed_dimension)

reduction = original_dimension - compressed_dimension

print("Reduced dimensions:", reduction)
```

```
Original dimension: 16384
Compressed dimension: 10280
Reduced dimensions: 6104
```

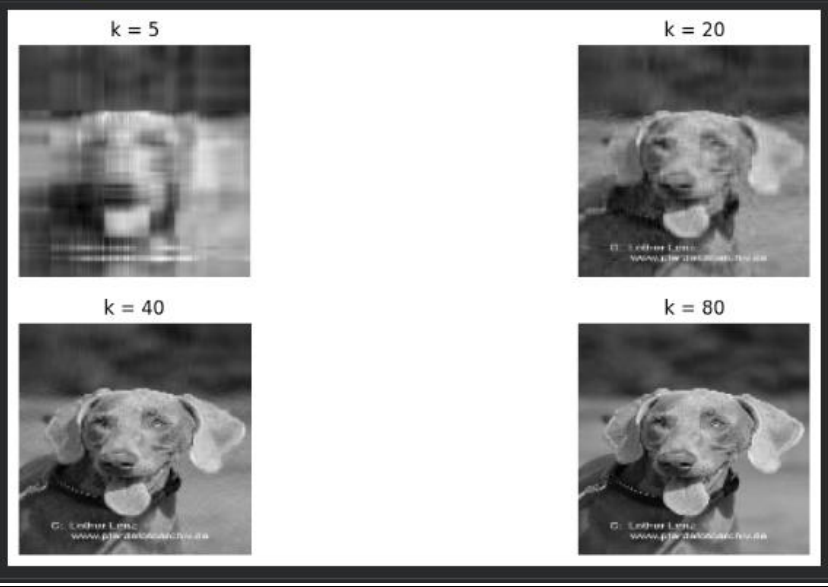
10. Calculated Compressed ratio

```
compression_ratio = original_dimension / compressed_dimension

print("Compression Ratio:", compression_ratio)
```

```
Compression Ratio: 1.593774319066148
```

11. Compared different K values

```
k_values = [5, 20, 40, 80]

plt.figure(figsize=(12,6))

for i,k in enumerate(k_values):

    Uk = U[:, :k]
    Sigmak = Sigma[:k, :k]
    Vk = V[:, :k]

    reconstructed = np.dot(Uk, np.dot(Sigmak, Vk.T))

    plt.subplot(2,2,i+1)
    plt.imshow(reconstructed, cmap='gray')
    plt.title("k = " + str(k))
    plt.axis("off")

plt.show()
```



Conclusion	Successfully performed SVD on an Image Dataset